[15] The implementation of `classifyPoints` is omitted: it can be derived from listing 12.3 by changing the condition checked, remembering that we don't look for the closest centroid in this case, but need all centroids within a certain distance. Also, this function would be the right point where the issue with unassigned points could be addressed: remember that the easiest, one-fits-all solution can be using a threshold distance equal to $T_1+T_2$.

[16] https://mahout.apache.org/docs/latest/algorithms/clustering/canopy/

[17] See section 12.3.1 for a definition of reachability and some examples.

[18] For a formal proof and a detailed description of the algorithm, see He, Yaobin, et al. "MR-DBSCAN: a scalable MapReduce-based DBSCAN algorithm for heavily skewed data." Frontiers of Computer Science 8.1 (2014): 83-99.

[19] $C_6$ doesn't qualify to merge with $C_1$, because none of its points in the margin region are core points; therefore, we need to explicitly perform some extra check (for instance, for each pair of clusters, verify if one is a subset of the other) to recognize these situations.

[20] It wouldn't make sense to have a shared object in the MapReduce model! Can you explain why? (hint: besides technical challenges, do we really want to introduce shared state?)

[21] https://en.wikipedia.org/wiki/Facade_pattern

[22] Since cells are overlapping, a point can be assigned to more than one cell.